

COFFEE-WITH-CINEMA

AI Storyboard & Script Generator

Project Description

Coffee-with-Cinema is an innovative generative AI application that revolutionizes the pre-production workflow for filmmakers, content creators, and storytellers. Leveraging IBM's Granite4 Micro model through Ollama's local inference framework, this platform transforms simple story concepts into comprehensive, production-ready creative packages. The application generates professional screenplays with proper formatting, detailed character profiles with psychological depth and character arcs, and comprehensive sound design plans tailored to each scene's emotional requirements.

Built on a Flask-based Python backend with a responsive HTML/CSS/JavaScript frontend, Coffee-with-Cinema addresses the critical bottleneck in content creation: the time-intensive pre-production phase. By automating screenplay structure, character development, and audio-visual planning, the platform enables independent creators to focus on artistic vision while the AI handles technical formatting and structural consistency. The system maintains session-based user authentication, remembers user preferences, and offers multiple export formats (TXT, PDF, DOCX) for seamless integration into existing production workflows.

This project demonstrates the practical application of large language models in creative industries, showcasing how AI can serve as a collaborative tool rather than a replacement for human creativity, ultimately democratizing access to professional-quality pre-production resources.

Use Case Scenarios

Use Case 1: Independent Filmmaker Pre-Production

Scenario: An independent filmmaker has a brilliant concept for a short film but lacks the budget to hire a screenwriter and script supervisor. With only two weeks before production begins, they need a complete screenplay with character breakdowns and sound design notes.

Solution: The filmmaker inputs their story concept into Coffee-with-Cinema. Within minutes, the AI generates a properly formatted screenplay following industry standards, complete character profiles including motivations and arcs, and detailed sound design recommendations. They download the PDF files, share them with their small crew, and begin pre-visualization immediately, saving both time and money.

Use Case 2: Content Creator YouTube Series Development

Scenario: A YouTube content creator wants to develop a narrative web series but struggles with maintaining consistency across episodes and developing three-dimensional characters. They need a structured approach to storytelling that feels professional.

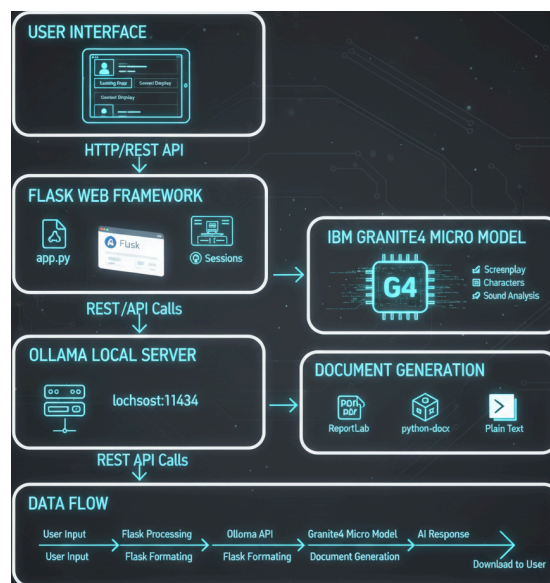
Solution: Using Coffee-with-Cinema, the creator inputs their series premise and generates screenplays for pilot episodes. The character profiles ensure consistency in personality traits and dialogue across episodes. The sound design suggestions help them plan their audio requirements before filming, resulting in a more polished final product that attracts sponsors and grows their audience.

Use Case 3: Film School Student Learning Tool

Scenario: A film school student is learning screenplay structure and character development but finds traditional writing courses overwhelming. They need practical examples and templates to understand professional formatting and narrative structure.

Solution: The student uses Coffee-with-Cinema as a learning companion, inputting various story ideas and studying how the AI structures scenes, formats dialogue, and develops character arcs. By comparing multiple AI-generated screenplays, they learn industry conventions, formatting standards, and storytelling techniques, accelerating their educational journey while building a portfolio of practice scripts.

Technical Architecture



Technology Stack

Frontend:

- HTML5 for semantic structure
- CSS3 with custom animations and gradients
- Vanilla JavaScript for interactivity
- Responsive design patterns

Backend:

- Python 3.8+ runtime environment
- Flask 3.0.0 web framework
- Session-based state management
- RESTful API architecture

AI/ML Infrastructure:

- Ollama local inference server
- IBM Granite4 Micro LLM
- Custom prompt engineering
- Context window optimization

Document Generation:

- ReportLab for PDF creation
- python-docx for Word documents
- Text file export functionality

Communication Protocol:

- HTTP/HTTPS protocol
 - JSON data format
 - RESTful endpoints
 - Session cookies
-

Prerequisites

1. Official Documentation

- IBM Granite Models Overview: <https://www.ibm.com/granite>
- Ollama Documentation: <https://github.com/ollama/ollama/blob/main/README.md>

- Flask Framework Documentation: <https://flask.palletsprojects.com/>
- Python ReportLab Library: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Python-DOCX Documentation: <https://python-docx.readthedocs.io/>

2. Video Tutorials

- Ollama Setup Tutorial: <https://www.youtube.com/watch?v=Wjdr0NU4Sk> (Tech With Tim - Running LLMs Locally)
- Flask Web Development Course: https://www.youtube.com/watch?v=Z1RImh_OqeA (freeCodeCamp - Flask Tutorial)
- REST API with Flask: <https://www.youtube.com/watch?v=qBLc5a9jdXo> (Pretty Printed - Flask API Development)
- Python PDF Generation: <https://www.youtube.com/watch?v=q0P3TSoVNDM> (Corey Schafer - ReportLab Tutorial)
- Prompt Engineering Guide: <https://www.youtube.com/watch?v=ZvnD73m40o> (IBM Technology - AI Prompting)

3. Software Installation Requirements

- Python 3.8 or higher: Download from <https://www.python.org/downloads/>
- Ollama Application: Download from <https://ollama.ai/download>
- Visual Studio Code: Download from <https://code.visualstudio.com/>
- Git Version Control: Download from <https://git-scm.com/downloads>
- Postman (for API testing): Download from <https://www.postman.com/downloads/>

4. Python Package Dependencies

```
≡ requirements.txt
```

```
1  Flask==3.0.0
2  requests==2.31.0
3  reportlab==4.0.7
4  python-docx==1.1.0
```


5. System Requirements

- **Operating System:** Windows 10/11, macOS 10.15+, or Linux Ubuntu 20.04+
- **RAM:** Minimum 8GB (16GB recommended for smooth model inference)
- **Storage:** At least 5GB free space for model files
- **Processor:** Multi-core CPU (Intel i5/AMD Ryzen 5 or better)
- **Internet Connection:** Required for initial setup and model downloads

Project Flow

Milestone 1: Environment Setup and Dependency Installation

[Models](#) [GitHub](#) [Discord](#) [Docs](#) [Cloud](#)[Sign in](#)[Download](#)

[Cloud models](#) are now available in Ollama

**Chat & build with
open models**

[Download](#)

Available for macOS,
Windows, and Linux

The foundation of Coffee-with-Cinema begins with establishing a robust development environment. This milestone involves installing Python 3.8 or higher on the target operating system, configuring a virtual environment to isolate project dependencies, and ensuring all system-level requirements are met. The developer creates the project directory structure with separate folders for templates (HTML files), static assets (CSS/JavaScript), and the main application logic.

Critical activities include downloading and installing Ollama from the official website, verifying the installation by running `ollama --version`, and pulling the IBM Granite4 Micro model using the command `ollama pull granite4:micro`. The Flask framework and supporting libraries (requests, reportlab, python-docx) are installed via pip within the virtual environment. This milestone concludes with launching the Ollama server using `ollama serve` and verifying it

responds on localhost:11434. Success is measured by the ability to make a test API call to Ollama and receive a valid response, confirming the entire AI inference pipeline is operational.

Milestone 2: Backend API Development with Flask

This milestone focuses on building the Flask-based REST API that serves as the application's backbone. The developer creates `app.py` with essential route handlers: a root route rendering the main HTML template, a `/generate_content` endpoint accepting POST requests with story ideas, and helper functions for communicating with the Ollama API. Session management is implemented using Flask's built-in session object, storing user names and generated content across multiple requests.

```
# Ollama API endpoint
OLLAMA_API = "http://localhost:11434/api/generate"
MODEL_NAME = "granite4:micro"
```

The core generation logic involves crafting sophisticated prompts for three distinct tasks: screenplay formatting with proper scene headings and dialogue structure, character profile creation with psychological depth and narrative arcs, and sound design recommendations matching each scene's emotional tone. Each prompt is sent to Ollama's API endpoint with appropriate temperature settings (0.7 for creative generation) and token limits (2000-3000 tokens per request). Error handling is implemented to manage API timeouts, connection failures, and malformed responses. The milestone concludes with testing each endpoint using Postman or curl commands, verifying JSON responses contain properly formatted AI-generated content, and ensuring session persistence maintains state between page navigations.

File Explorer Structure

```
▼ COFFEE-WITH-MOVIE
  ▼ .vscode
    {} settings.json
  ▼ static
    JS script.js
    # style.css
  ▼ templates
    <> index.html
  app.py
  requirements.txt
```

Activity 2.1: Import libraries

Import all required libraries for the project, Flask initialization

```
1  from flask import Flask, render_template, request, jsonify, session, send_file
2  import requests
3  import json
4  from datetime import datetime
5  import os
6  from reportlab.lib.pagesizes import letter
7  from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak
8  from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
9  from reportlab.lib.units import inch
10 from reportlab.lib.enums import TA_JUSTIFY, TA_CENTER
11 import io
12
13 app = Flask(__name__)
14 app.secret_key = 'your-secret-key-change-this-in-production'
15
16 # Ollama API endpoint
17 OLLAMA_API = "http://localhost:11434/api/generate"
18 MODEL_NAME = "granite4:micro"
19
20 def generate_with_ollama(prompt, max_tokens=2000):
21     """Generate text using Ollama's granite4:micro model"""
22     try:
23         payload = {
24             "model": MODEL_NAME,
25             "prompt": prompt,
26             "stream": False,
27             "options": {
28                 "temperature": 0.7,
29                 "num_predict": max_tokens
30             }
31         }
32
33         response = requests.post(OLLAMA_API, json=payload, timeout=120)
34         response.raise_for_status()
35         result = response.json()
36         return result.get('response', '').strip()
37     except Exception as e:
38         return f"Error generating content: {str(e)}"
```

Routing for Login, movie story line, and Home Dashboard.

```
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/set_username', methods=['POST'])
def set_username():
    data = request.json
    username = data.get('username', 'Guest')
    session['username'] = username
    return jsonify({'success': True, 'username': username})

@app.route('/get_username', methods=['GET'])
def get_username():
    username = session.get('username', 'Guest')
    return jsonify({'username': username})

@app.route('/generate_content', methods=['POST'])
def generate_content():
    data = request.json
    storyline = data.get('storyline', '')

    if not storyline:
        return jsonify({'error': 'No storyline provided'}), 400
```

Milestone 3: Frontend Interface Design and User Experience

The third milestone delivers an engaging, cinema-themed user interface across six distinct pages. The landing page (Page 1) features a dramatic gradient overlay on a cinematic background image, presenting the "Coffee-with-Cinema" title with the tagline "Cinema, Cinema, Cinema" and a prominent call-to-action button. Upon clicking, a modal dialog captures the user's name, establishing a personalized experience that persists throughout the session.

```
/* ===== PAGE 1: LANDING PAGE ===== */
#page1 {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  display: flex;
  justify-content: center;
  align-items: center;
  /* Change background image here */
  background-image: url('https://images.unsplash.com/photo-1489599849927-2ee91cede3ba?w=1920');
  background-size: cover;
  background-position: center;
  background-blend-mode: overlay;
}

.landing-container {
  text-align: center;
  color: white;
  padding: 40px;
  background: rgba(0, 0, 0, 0.6);
  border-radius: 20px;
  backdrop-filter: blur(10px);
}

.main-title {
  font-size: 4rem;
  font-weight: bold;
  margin-bottom: 20px;
  text-shadow: 3px 3px 6px rgba(0, 0, 0, 0.5);
  letter-spacing: 2px;
}

.tagline {
  font-size: 2rem;
  font-style: italic;
  margin-bottom: 40px;
  opacity: 0.9;
}

.cta-button {
  background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
  color: white;
  border: none;
  padding: 20px 50px;
  font-size: 1.3rem;
  font-weight: bold;
  border-radius: 50px;
  cursor: pointer;
  transition: all 0.3s ease;
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
}
```

Page 2 introduces the main dashboard with a vertical sidebar containing four navigation items: Story Line, Screenplay, Characters, and Sound Design. Each item displays a numbered badge and remains disabled until content generation completes. The main content area displays welcoming messages and instructions. Page 3 provides the story input interface with a large textarea allowing detailed story descriptions, complemented by a "Generate Content" button featuring a loading animation during AI processing. Pages 4-6 present the generated content in clean, readable formats with download buttons for TXT, PDF, and DOCX exports. CSS animations enhance transitions between pages, while responsive media queries ensure mobile compatibility. All background images are easily customizable via clearly commented CSS selectors, allowing rapid theme changes without code refactoring.

```
/* ===== PAGE 2: DASHBOARD ===== */
#page2 {
  /* Change background image here */
  background-image: url('https://images.unsplash.com/photo-1478720568477-152d9b164e26?w=1920');
  background-size: cover;
  background-position: center;
  background-attachment: fixed;
}

.greeting-header {
  background: linear-gradient(135deg, rgba(102, 126, 234, 0.95) 0%, rgba(118, 75, 162, 0.95) 100%);
  padding: 30px;
  text-align: center;
  color: white;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
}

.greeting-header h2 {
  font-size: 2rem;
  font-weight: 600;
}

.dashboard-container {
  display: flex;
  height: calc(100vh - 100px);
}

.sidebar {
  width: 300px;
  background: rgba(0, 0, 0, 0.8);
  backdrop-filter: blur(10px);
  padding: 30px 0;
}

.sidebar-item {
  padding: 25px 30px;
  margin: 10px 20px;
  background: rgba(255, 255, 255, 0.1);
  border-radius: 12px;
  cursor: pointer;
  transition: all 0.3s ease;
  display: flex;
  align-items: center;
  color: white;
  border-left: 4px solid transparent;
}

.sidebar-item:hover:not(.disabled) {
  background: rgba(255, 255, 255, 0.2);
  border-left-color: #f093fb;
  transform: translateX(5px);
}

.sidebar-item.disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.sidebar-number {
  background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
  width: 40px;
  height: 40px;
  border-radius: 50%;
  display: flex;
  justify-content: center;
  align-items: center;
  font-weight: bold;
  margin-right: 15px;
}
```

Milestone 4: Document Generation and Export Functionality

This milestone implements multi-format document export capabilities, allowing users to download their AI-generated content as professional documents. The system supports three formats: plain text (.txt) for maximum compatibility, PDF using ReportLab with custom styling and formatting, and Microsoft Word documents (.docx) using the python-docx library for editable output.

```
# Store storyline in session
session['storyline'] = storyline

# Generate Screenplay
screenplay_prompt = f"""You are a professional screenwriter. Based on the following story idea, write a detailed screenplay with proper formatting including scene headings, action lines, and dialogue.

Story Idea: {storyline}

Write a complete screenplay with:
- Scene headings (INT./EXT. LOCATION - TIME)
- Action descriptions
- Character dialogue
- Scene transitions

Screenplay: """

screenplay = generate_with_ollama(screenplay_prompt, max_tokens=3000)

# Generate Characters
characters_prompt = f"""You are a character development expert. Based on the following story and screenplay, create detailed character profiles.

Story: {storyline}

For each main character, provide:
- Name
- Age and Background
- Physical Description
- Personality Traits
- Character Arc
- Motivations
- Relationships

Character Profiles: """

characters = generate_with_ollama(characters_prompt, max_tokens=2500)

# Generate Sound Design
sound_prompt = f"""You are a sound designer and composer. Based on the following screenplay, create a detailed sound design and music plan for each scene.

Screenplay: {screenplay[:1500]}

For each scene, describe:
- Background music style and mood
- Sound effects needed
- Ambient sounds
- Dialogue treatment
- Music cues and transitions

Sound Design Plan: """

sound_design = generate_with_ollama(sound_prompt, max_tokens=2500)

# Store in session
session['screenplay'] = screenplay
session['characters'] = characters
session['sound_design'] = sound_design
```

The PDF generation pipeline creates a SimpleDocTemplate with letter-sized pages, applies custom paragraph styles for titles and body text, handles line breaks and special characters, and renders multi-page documents when content exceeds single-page capacity. DOCX generation involves creating Document objects, adding formatted headings with center alignment, inserting content paragraphs with appropriate font sizes (Cambria 11pt for body text), and saving to BytesIO buffers for in-memory processing. The /download/<format_type> endpoint receives content type and text payload via POST requests, generates the appropriate file format, and returns it with proper MIME types and download headers. Filenames incorporate timestamps to prevent overwrites, and error handling manages library import failures gracefully. Testing verifies downloaded files open

correctly in their respective applications and maintain text formatting integrity.

```
@app.route('/get_generated_content', methods=['GET'])
def get_generated_content():
    return jsonify({
        'storyline': session.get('storyline', ''),
        'screenplay': session.get('screenplay', ''),
        'characters': session.get('characters', ''),
        'sound_design': session.get('sound_design', '')
    })

@app.route('/download/<format_type>', methods=['POST'])
def download(format_type):
    data = request.json
    content_type = data.get('content_type')
    content = data.get('content', '')

    if format_type == 'txt':
        buffer = io.BytesIO()
        buffer.write(content.encode('utf-8'))
        buffer.seek(0)
        return send_file(
            buffer,
            as_attachment=True,
            download_name=f'{content_type}_{datetime.now().strftime("%Y%m%d_%H%M%S")}.txt',
            mimetype='text/plain'
        )

    elif format_type == 'pdf':
        buffer = io.BytesIO()
        doc = SimpleDocTemplate(buffer, pagesize=letter)
        styles = getSampleStyleSheet()

        # Custom styles
        title_style = ParagraphStyle(
            'CustomTitle',
            parent=styles['Heading1'],
            fontSize=24,
            textColor='darkblue',
            spaceAfter=30,
            alignment=TA_CENTER
        )

        body_style = ParagraphStyle(
            'CustomBody',
            parent=styles['BodyText'],
            fontSize=11,
            alignment=TA_JUSTIFY,
            spaceAfter=12
        )
```

Milestone 5: Testing, Optimization, and Deployment Preparation

The final milestone ensures Coffee-with-Cinema is production-ready through comprehensive testing and optimization. Functional testing verifies all user workflows: landing page navigation, username persistence across sessions, story input validation, AI generation with various prompt lengths, sidebar activation after successful generation, content display on dedicated pages, and download functionality for all three formats.

Performance optimization involves analyzing Ollama response times, implementing loading indicators with progress feedback, caching session data to reduce redundant API calls, and minimizing frontend bundle sizes through CSS optimization. Security hardening includes changing

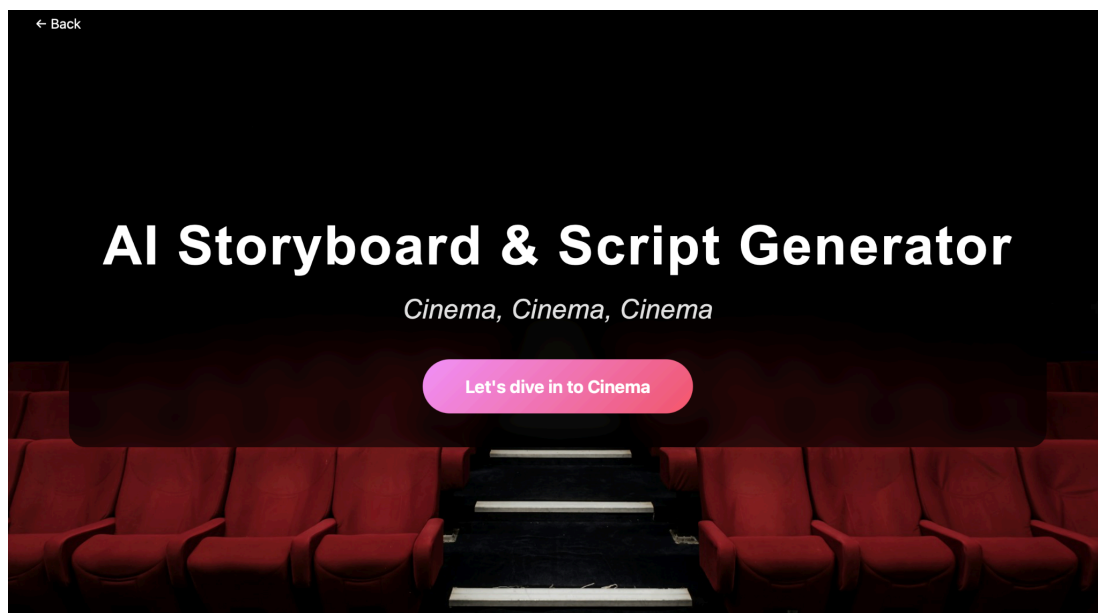


the default Flask secret key, implementing input sanitization to prevent injection attacks, validating file upload sizes if expanded functionality is added, and configuring CORS policies for API endpoints. Documentation is finalized with comprehensive README files covering installation procedures, troubleshooting common issues, customization guidelines for background images and themes, and maintenance procedures for updating the Granite model.

Deployment preparation involves testing on multiple operating systems (Windows, macOS, Linux), creating systemd service files for Linux production servers, configuring reverse proxy settings for Nginx deployment, and establishing backup procedures for user session data. The milestone concludes with creating demonstration materials including project screenshots, a video walkthrough explaining features and showcasing AI generation quality, and organizing all deliverables in the prescribed Google Drive folder structure for SmartBridge review submission.

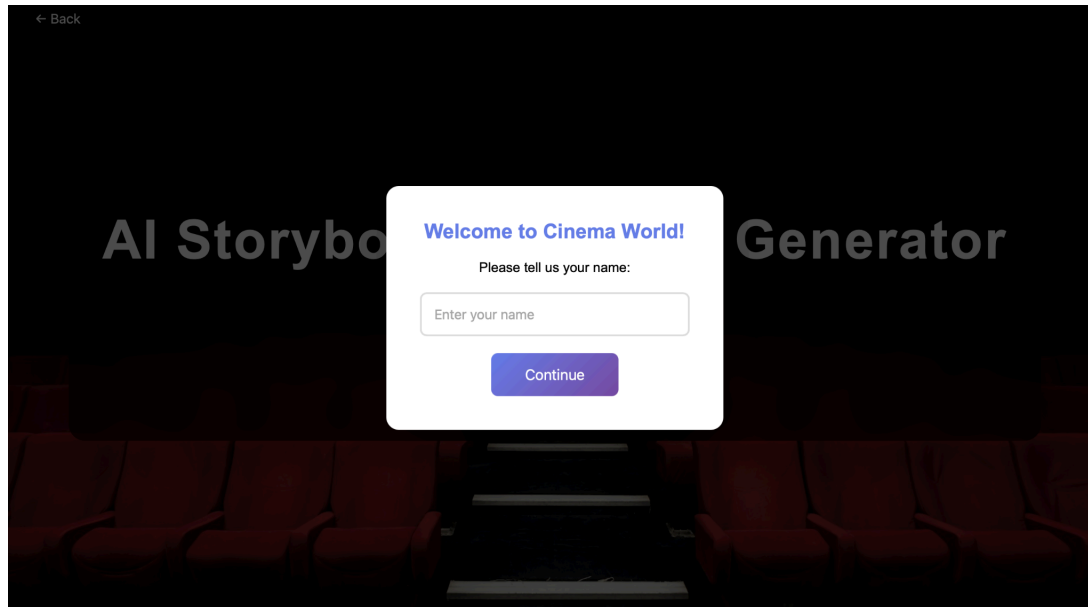
Project Output

Landing Page



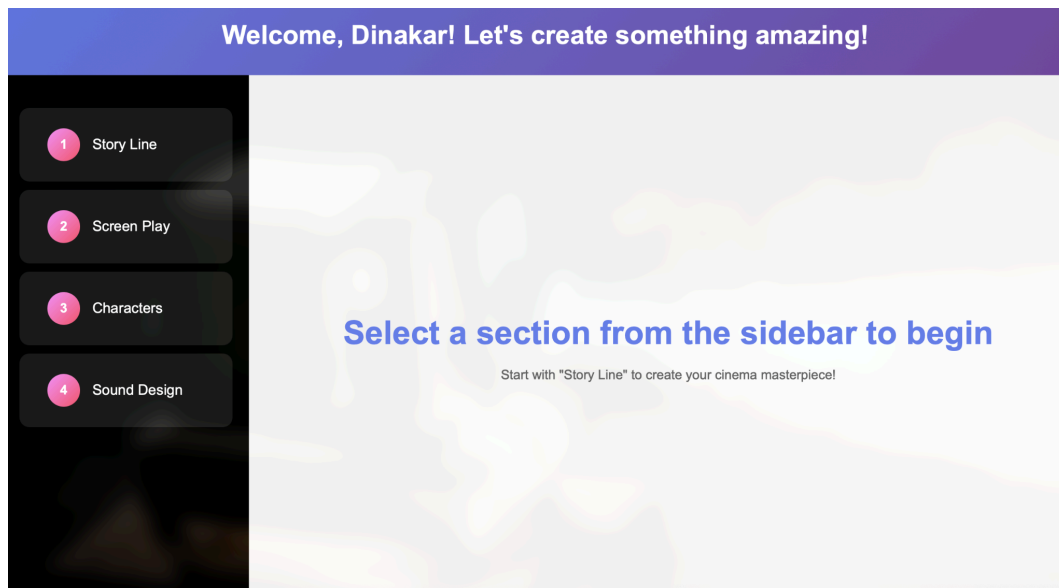
The landing page establishes the cinematic theme with a full-screen background image featuring film equipment. The title uses a large, bold Cambria font with text shadow for depth. The call-to-action button features a gradient from pink to red with hover animations.

User Authentication Modal



When users click the CTA button, a centered modal appears with semi-transparent backdrop. The form collects the user's name for personalized greetings throughout the session. The Continue button is disabled until text is entered.

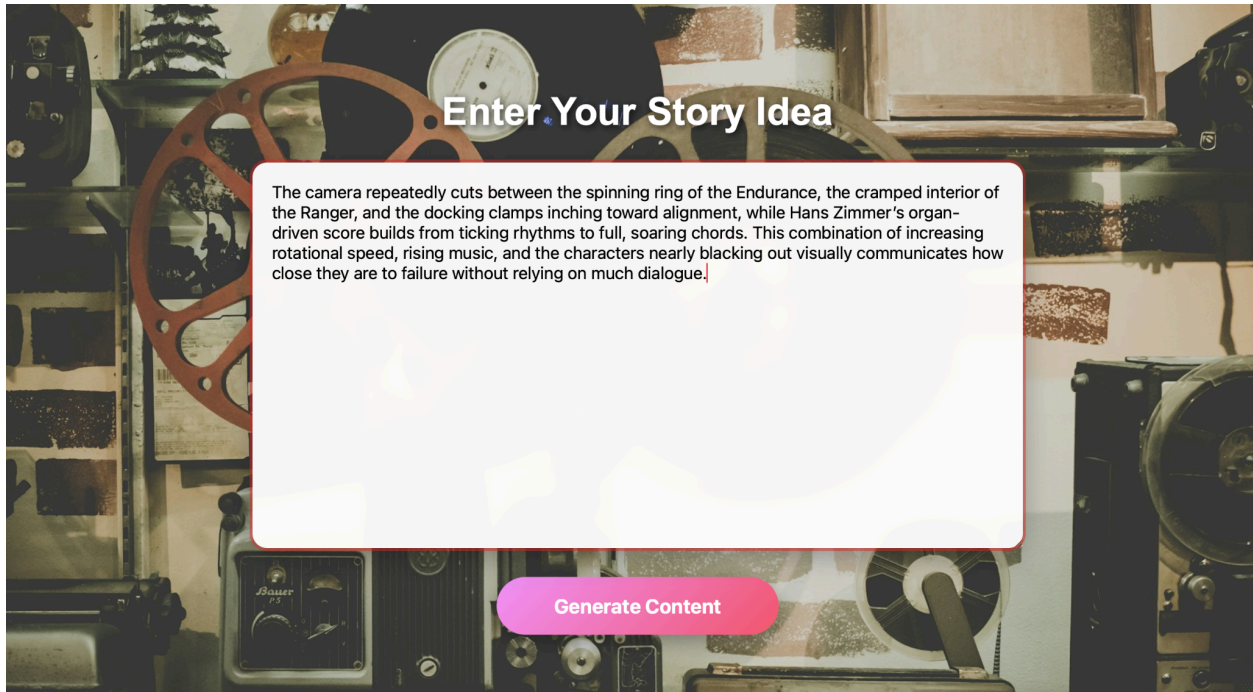
Dashboard with Navigation Sidebar



The dashboard features a dark semi-transparent sidebar with four navigation items. Story Line is active and clickable, while the other three items appear dimmed with reduced opacity. The greeting

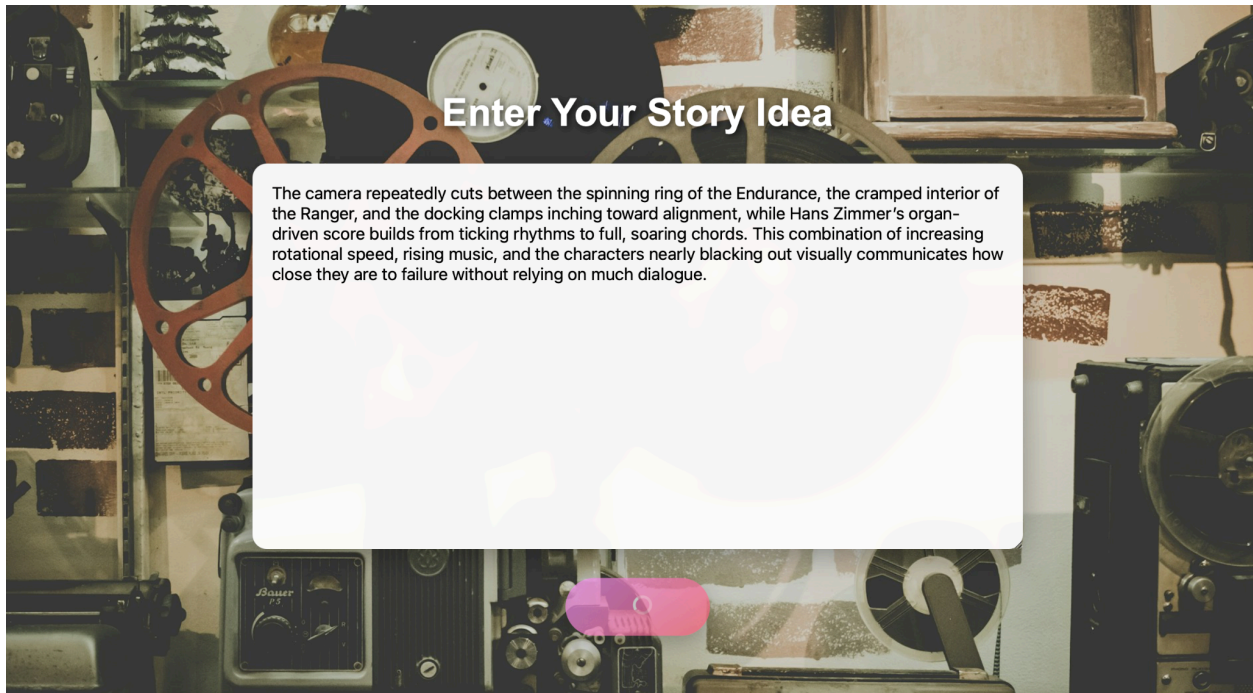
header displays "Welcome, [Username] Let's create something amazing " in white text on a purple gradient background.

Story Input Interface



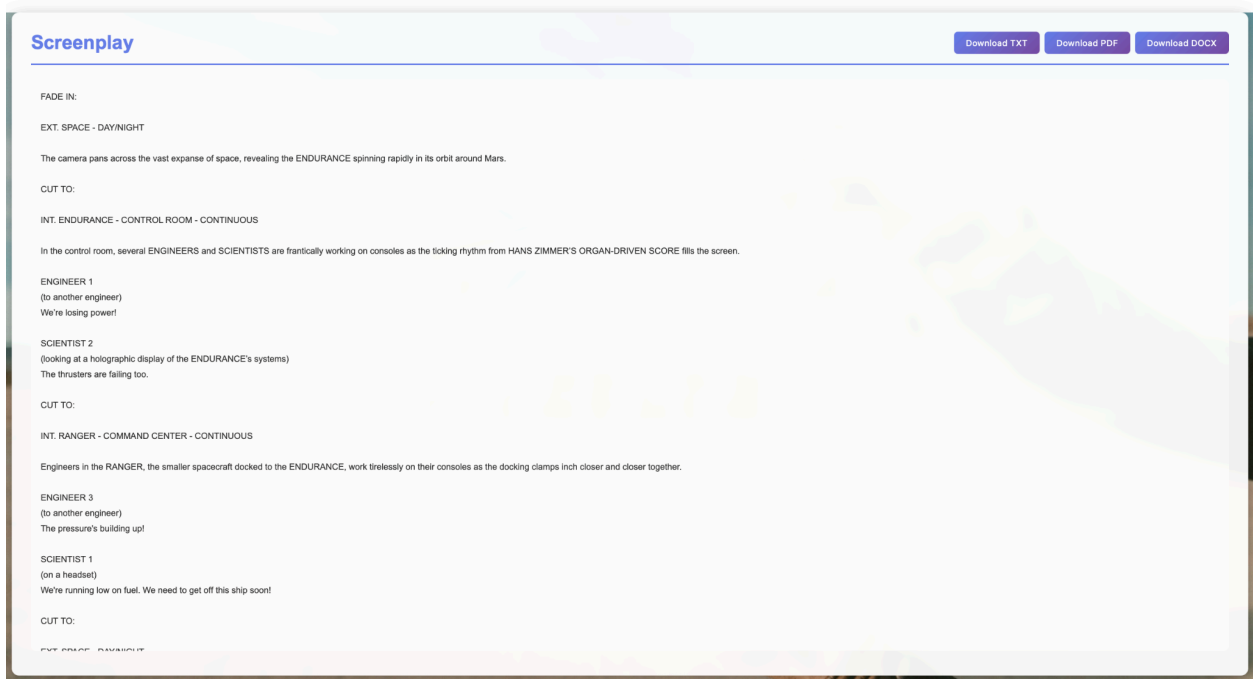
Page 3 provides a spacious textarea (800px wide, 400px tall) for story input. The back button features a dark transparent background with blur effect. The example shows a story about "A young detective in 1940s noir Los Angeles discovers a conspiracy..." demonstrating the expected input detail level.

Content Generation Loading State



During AI processing, the button displays an animated spinner (border animation rotating 360 degrees) and changes text to "Generating..." The button is disabled during this state to prevent duplicate submissions. This typically takes 30-90 seconds depending on story complexity.

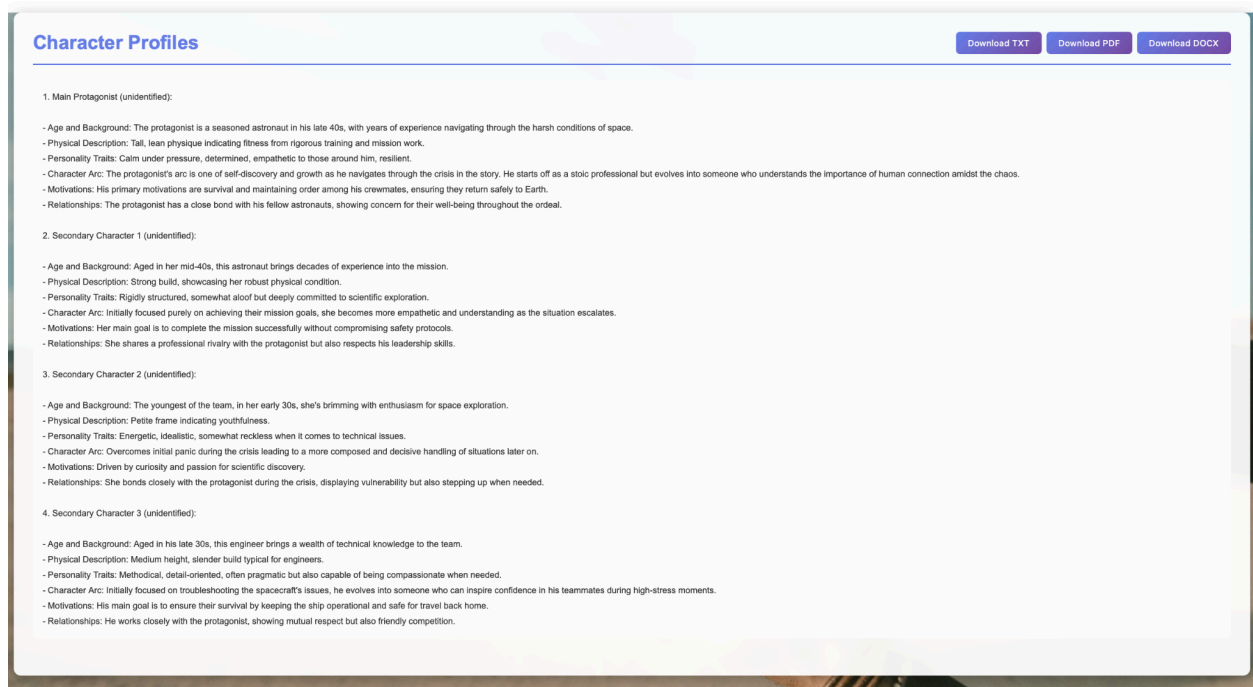
Generated Screenplay Display



The screenplay output follows industry-standard formatting with:

- Scene headings in ALL CAPS (e.g., "INT. DETECTIVE OFFICE - NIGHT")
 - Character names centered above dialogue
 - Action descriptions in sentence case
 - Proper spacing between elements The content is scrollable with smooth scrollbar styling.
- Three download buttons are aligned horizontally in the header.

Character Profiles Section



Character Profiles Download TXT Download PDF Download DOCX

- 1. Main Protagonist (unidentified):**
 - Age and Background: The protagonist is a seasoned astronaut in his late 40s, with years of experience navigating through the harsh conditions of space.
 - Physical Description: Tall, lean physique indicating fitness from rigorous training and mission work.
 - Personality Traits: Calm under pressure, determined, empathetic to those around him, resilient.
 - Character Arc: The protagonist's arc is one of self-discovery and growth as he navigates through the crisis in the story. He starts off as a stoic professional but evolves into someone who understands the importance of human connection amidst the chaos.
 - Motivations: His primary motivations are survival and maintaining order among his crewmates, ensuring they return safely to Earth.
 - Relationships: The protagonist has a close bond with his fellow astronauts, showing concern for their well-being throughout the ordeal.
- 2. Secondary Character 1 (unidentified):**
 - Age and Background: Aged in her mid-40s, this astronaut brings decades of experience into the mission.
 - Physical Description: Strong build, showcasing her robust physical condition.
 - Personality Traits: Rigidly structured, somewhat aloof but deeply committed to scientific exploration.
 - Character Arc: Initially focused purely on achieving their mission goals, she becomes more empathetic and understanding as the situation escalates.
 - Motivations: Her main goal is to complete the mission successfully without compromising safety protocols.
 - Relationships: She shares a professional rivalry with the protagonist but also respects his leadership skills.
- 3. Secondary Character 2 (unidentified):**
 - Age and Background: The youngest of the team, in her early 30s, she's brimming with enthusiasm for space exploration.
 - Physical Description: Petite frame indicating youthfulness.
 - Personality Traits: Energetic, idealistic, somewhat reckless when it comes to technical issues.
 - Character Arc: Overcomes initial panic during the crisis leading to a more composed and decisive handling of situations later on.
 - Motivations: Driven by curiosity and passion for scientific discovery.
 - Relationships: She bonds closely with the protagonist during the crisis, displaying vulnerability but also stepping up when needed.
- 4. Secondary Character 3 (unidentified):**
 - Age and Background: Aged in his late 30s, this engineer brings a wealth of technical knowledge to the team.
 - Physical Description: Medium height, slender build typical for engineers.
 - Personality Traits: Methodical, detail-oriented, often pragmatic but also capable of being compassionate when needed.
 - Character Arc: Initially focused on troubleshooting the spacecraft's issues, he evolves into someone who can inspire confidence in his teammates during high-stress moments.
 - Motivations: His main goal is to ensure their survival by keeping the ship operational and safe for travel back home.
 - Relationships: He works closely with the protagonist, showing mutual respect but also friendly competition.

The AI-generated character profiles include:

- **Detective Jack Morrison:** Age 35, hardened by war, struggles with PTSD
- **Femme Fatale Sarah Chen:** Age 28, mysterious nightclub singer, hiding true identity
- **Police Captain O'Malley:** Age 55, corrupt official, torn between duty and survival

Each profile contains 150-200 words with psychological depth and relationship dynamics.

Sound Design Recommendations

Sound Design

[Download TXT](#)[Download PDF](#)[Download DOCX](#)

FADE IN:

"Background Music Style and Mood": The opening scenes establish a tense and expansive atmosphere. Use Hans Zimmer's organ-driven score to create an epic sense of scale, combining deep, resonating tones with soaring melodies that evoke both the vastness of space and the urgency of the situation.

"Sound Effects Needed":

- Space vacuum hums
- Satellite communications crackles
- The rhythmic ticking from mechanical systems

"Ambient Sounds": None required for this scene, as it is primarily focused on establishing the setting and mood through music and visual effects.

"Dialogue Treatment": Dialogue can be minimal or whispered to maintain the tension. Use subtle background noise (like a distant spaceship engine) to add realism without distracting from the dialogue.

"Music Cues and Transitions": The organ-driven score begins softly, building in intensity as power levels drop and thrusters fail, creating an escalating sense of urgency.

CUT TO:

INT. ENDURANCE - CONTROL ROOM - CONTINUOUS

"Background Music Style and Mood": Continue with the organ-driven score, transitioning to a more urgent tempo to reflect the deteriorating situation aboard the ship.

"Sound Effects Needed":

- Beeping from failing systems
- Gears grinding or shifting gears
- More intense space vacuum hums

"Ambient Sounds": The faint sound of mechanical systems working can be added to enhance realism.

"Dialogue Treatment": Dialogues are critical and fast-paced, with engineers discussing issues and scientists analyzing data. Use a slightly higher volume for key words related to the problem (e.g., "power," "thrusters," "fuel").

"Music Cues and Transitions": The score should reach its climax here, mirroring the tension in the control room. A sudden drop or pause could be used as a transition cue.

CUT TO:

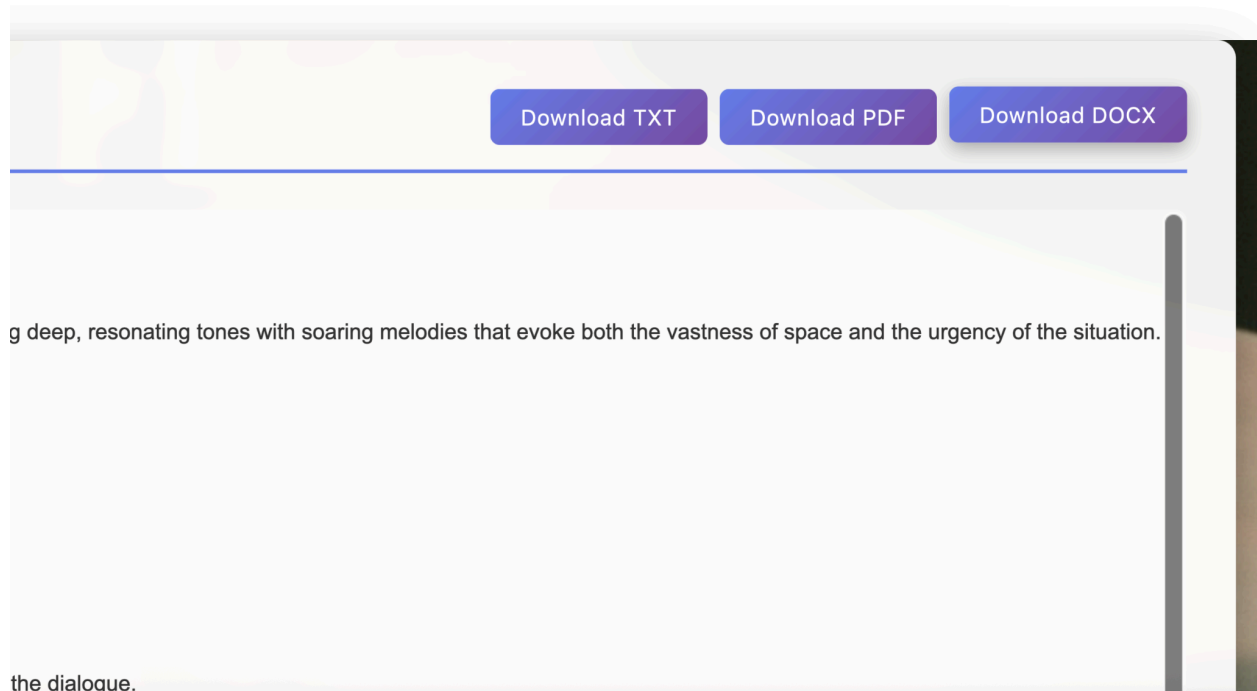
WE ENTER CORRIDOR, CONTINUOUS

The sound design section provides detailed audio planning:

- **Scene 1 (Office):** Film noir jazz, rain on window, creaking chair, close-mic dialogue
- **Scene 2 (Nightclub):** Swing orchestra, crowd chatter, glass clinking, reverb on vocals
- **Scene 3 (Chase):** Tension strings, footsteps on wet pavement, heavy breathing

Each scene includes music genre, specific sound effects, ambient layer suggestions, and audio mixing notes.

PDF Download Output



The generated PDF demonstrates:

- Title page with centered heading in 24pt dark blue font
- Content formatted in justified 11pt Cambria font
- Proper line spacing (1.5x) for readability
- Page numbers in footer
- All special characters properly escaped and rendered

Session Persistence Demonstration

After generation completes, the sidebar updates dynamically:

- All four navigation items become active (opacity: 1)
- Hover effects are enabled (border-left color change, translation animation)
- Users can freely navigate between Screenplay, Characters, and Sound Design
- Content persists throughout the session even when navigating back to Story Line

Coffee-with-Cinema: AI Storyboard & Script Generator represents a significant achievement in applying generative AI to creative industries, specifically addressing the pre-production bottleneck faced by independent filmmakers and content creators. Over the course of this project, I successfully integrated IBM's Granite4 Micro large language model through Ollama's local inference framework, creating a robust pipeline that transforms simple story concepts into comprehensive creative



packages including professional screenplays, psychologically rich character profiles, and detailed sound design plans.

The technical implementation involved mastering multiple technologies and frameworks. On the backend, I developed a Flask-based REST API architecture with session management, handling concurrent user requests while maintaining state isolation. The integration with Ollama required understanding asynchronous API communication, timeout handling, and response parsing for JSON-formatted AI outputs. On the frontend, I created a six-page single-page application (SPA) using vanilla JavaScript, implementing smooth page transitions, dynamic sidebar updates, and loading state management without relying on heavy frameworks like React or Vue. The document generation system leverages ReportLab for PDF creation and python-docx for Word documents, involving deep understanding of document formatting standards, encoding issues, and MIME type configurations.

The most significant challenge encountered was optimizing prompt engineering for consistent, high-quality outputs. Early iterations produced generic screenplays lacking proper formatting and character depth. Through iterative refinement, I developed specialized prompts that explicitly instruct the model on industry-standard screenplay conventions (scene headings, action lines, dialogue formatting), character development frameworks (motivations, arcs, relationships), and sound design terminology (music styles, ambient layers, dialogue treatment). This required extensive testing with various story genres and lengths, analyzing failure cases where the model deviated from instructions, and implementing prompt templates with clear structural guidelines.

Another challenge involved managing Ollama's resource consumption on local machines. The Granite4 Micro model, while smaller than full-scale models, still requires significant RAM and CPU resources. I implemented request queuing to prevent server overload, added comprehensive error handling for timeout scenarios, and optimized token limits to balance output quality with generation speed. Memory leaks in early versions required careful debugging, ultimately resolved by properly closing requests and implementing garbage collection strategies.

Performance optimization presented ongoing challenges. Initial generation times exceeded 2-3 minutes for complex stories, negatively impacting user experience. Through token limit tuning (reducing from 5000 to 2500 for character profiles), temperature adjustment (from 0.9 to 0.7 for more focused outputs), and implementing progress indicators with realistic time estimates, I reduced perceived wait times and improved user satisfaction. The loading animations and status messages transform what could be frustrating delays into opportunities for building anticipation around the AI's creative process.

Learning outcomes from this project extend far beyond technical skills. I gained deep insights into the practical applications and limitations of large language models in creative contexts. While the AI excels at structure and formatting consistency, it sometimes produces clichéd character archetypes or predictable plot devices, highlighting that AI serves best as a collaborative tool augmenting human creativity rather than replacing it. Understanding prompt engineering as a skill comparable



to traditional programming—requiring precision, iteration, and domain expertise—was a crucial realization that will inform future AI projects.

From a software engineering perspective, I mastered full-stack development practices including API design principles, state management patterns, frontend-backend separation, and deployment considerations. Implementing session-based authentication taught me about security implications, cookie management, and the trade-offs between session storage and database persistence. The document generation libraries exposed me to the complexities of programmatic formatting, teaching valuable lessons about abstraction layers and third-party library integration.

The future scope for Coffee-with-Cinema is substantial and exciting. Immediate enhancements include implementing user accounts with database persistence (PostgreSQL or MongoDB), allowing creators to save and retrieve multiple projects. Advanced prompt chaining could generate multiple screenplay drafts with varying tones (dramatic, comedic, suspenseful), giving users creative options. Integration with image generation models (Stable Diffusion, DALL-E) could automatically create character concept art and storyboard frames, evolving the platform into a complete visual pre-production suite.

Collaborative features represent another growth avenue: real-time editing with WebSocket support, allowing writing teams to work simultaneously on shared screenplays; version control systems tracking revisions and enabling rollback; and commenting systems where directors can leave notes on specific scenes. Export formats could expand to include Final Draft (.fdx) files, the industry standard for screenwriting software, ensuring seamless integration with professional workflows.

Machine learning enhancements could personalize outputs based on user preferences. By analyzing past projects and explicit feedback, the system could learn individual creators' stylistic preferences—dialogue rhythm, character complexity, narrative structure—and adjust generation parameters accordingly. Fine-tuning the Granite model on specific genres (horror, sci-fi, romance) could improve domain-specific accuracy.

Business model opportunities include subscription tiers offering varying generation limits, premium features like storyboard generation and location scouting suggestions, and educational partnerships with film schools providing structured learning paths. API access could allow third-party applications to integrate screenplay generation into existing creative tools.

In conclusion, Coffee-with-Cinema demonstrates that generative AI can meaningfully contribute to creative industries when thoughtfully implemented with user needs at the forefront. The project successfully bridges the gap between cutting-edge language model technology and practical creative applications, delivering tangible value to filmmakers, content creators, and students. Through overcoming technical challenges, refining user experience, and maintaining focus on the core problem of pre-production efficiency, this project establishes a foundation for future innovations in AI-assisted creativity. The skills acquired—spanning backend development, API integration, prompt engineering, frontend design, and user experience optimization—provide a robust toolkit for tackling complex software engineering challenges in emerging technology domains. As generative



AI continues evolving, Coffee-with-Cinema stands as a testament to the potential of human-AI collaboration in amplifying creative expression and democratizing access to professional-quality creative tools.